Window functions

SQL Queries

1. SELECT emp_name, department, salary, ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num FROM employees;

2. SELECT emp_name, department, salary, RANK() OVER (ORDER BY salary DESC) AS salary_rank FROM employees;

3. SELECT emp_name, department, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank FROM employees;

4. SELECT emp_name, department, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num FROM employees;

5. SELECT emp_name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank FROM employees;

6. SELECT emp_name, department, salary FROM (SELECT emp_name, department, salary, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees) e WHERE rnk <= 2;

7. SELECT emp_name, department, salary, FIRST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY salary DESC) AS highest_paid_employee FROM employees;

8. SELECT emp_name, department, salary, LAST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS lowest_paid_employee FROM employees;

9. SELECT emp_name, department, salary, FIRST_VALUE(salary) OVER (PARTITION BY department ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS highest_salary, LAST_VALUE(salary) OVER (PARTITION BY department ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS lowest_salary FROM employees;

10. SELECT emp_name, department, salary FROM (SELECT emp_name, department, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) e WHERE rnk = 2;

Coding Questions

1. Delete a Node from a BST

Solution :

```
        class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;

        if (key < root.val) {
            root.left = deleteNode(root.left, key);
        } else if (key > root.val) {
            root.right = deleteNode(root.right, key);
        } else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            TreeNode temp = root.right;

            while (temp.left != null) {
                temp = temp.left;
            }

            root.val = temp.val;
            root.right = deleteNode(root.right, temp.val);
        }

        return root;
    }
}
```

2. Check if Binary Tree is a BST

Solution :

```
        class Solution {
    public boolean isValidBST(TreeNode root) {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    boolean check(TreeNode root, long min, long max) {
        if (root == null) return true;

        if (root.val <= min || root.val >= max) {
            return false;
        }

        return check(root.left, min, root.val) &&
               check(root.right, root.val, max);
    }
}
```

3. Kth Smallest and Kth Largest in BST

Solution :

```
        class Solution {
    int count;
    int result;

    public int kthSmallest(TreeNode root, int k) {
        count = 0;
        result = -1;
        inorder(root, k);
        return result;
    }

    void inorder(TreeNode root, int k) {
        if (root == null) return;

        inorder(root.left, k);

        count++;

        if (count == k) {
            result = root.val;
            return;
        }

        inorder(root.right, k);
    }

    public int kthLargest(TreeNode root, int k) {
        count = 0;
        result = -1;
        reverseInorder(root, k);
        return result;
    }

    void reverseInorder(TreeNode root, int k) {
        if (root == null) return;

        reverseInorder(root.right, k);

        count++;

        if (count == k) {
            result = root.val;
            return;
        }

        reverseInorder(root.left, k);
    }
}
```

4. Check if BST is Balanced

Solution :

```
        class Solution {
    public boolean isBalanced(TreeNode root) {
        return height(root) != -1;
```

```
    }

    int height(TreeNode root) {
        if (root == null) return 0;

        int left = height(root.left);
        if (left == -1) return -1;

        int right = height(root.right);
        if (right == -1) return -1;

        if (Math.abs(left - right) > 1) {
            return -1;
        }

        return Math.max(left, right) + 1;
    }
}
```